if i wanted to add edge detection into my project, how difficult would that be? i dont have much time so it would need to be done similar to how ive done my bloom on my lightsaber blade where i have the multiple rendering passes within its own section, here is my scenebasic_uniform.cpp:

```
#include "scenebasic_uniform.h"

#include <cstdio>
#include <cstdlib>

#include <string>
using std::string;

#include <sstream>
#include <iostream>
using std::cerr;
using std::endl;

#include <GLFW/glfw3.h>
#include "helper/glutils.h"
#include "helper/texture.h"

using glm::vec3;
using glm::vec4;
using glm::mat3;
using glm::mat4;

SceneBasic_Uniform::SceneBasic_Uniform() : tPrev(0), angle(0.0f),
rotSpeed(glm::pi<float>() / 2.0f), plane(79.4f, 53.2f, 100, 100),
sky(100.0f) {
    LightsaberMesh = ObjMesh::load("media/Lightsaber_03.obj", true);
    BladeMEsh = ObjMesh::load("media/cylinder.obj", true);
}

void SceneBasic_Uniform::initScene()
{
    compile();
    prog.use();
    glEnable(GL_DEPTH_TEST);

    glEnable(GL_STENCIL_TEST);
    glClearStencil(0);

    model = mat4(1.0f);
    view = glm::lookAt(vec3(5.0f, 7.5f, 7.5f), vec3(0.0f, 0.75f, 0.0f),
vec3(0.0f, 1.0f, 0.0f));
    //view = glm::lookAt(camPos, camPos + camFront, camUp);

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        // Capture cursor
        glfwSetInputMode(win, GLFW_CURSOR, cursorCaptured ?
GLFW_CURSOR_DISABLED : GLFW_CURSOR_NORMAL);

        glfwGetCursorPos(win, &lastMouseX, &lastMouseY);
        firstMouse = true;
    }

    projection = mat4(1.0f);
    rotateModel = mat4(1.0f);
    rotateModel = glm::translate(rotateModel, vec3(4.0f, 0.26f, 5.0f));
    angle = 0.0f;


    //blinnPhongSpot stuff
    prog.setUniform("Spot.L", vec3(0.7f));
    prog.setUniform("Spot.La", vec3(0.5f));
    prog.setUniform("Spot.Exponent", 25.0f);
    prog.setUniform("Spot.Cutoff", glm::radians(20.0f));

    prog.setUniform("Light.L", vec3(0.5f));
    prog.setUniform("Light.La", vec3(0.25f));

    //fog
    prog.setUniform("Fog.MinDistance", 10.0f);
    prog.setUniform("Fog.MaxDistance", 45.0f);
    prog.setUniform("Fog.Colour", fogColour);


    cubeTex =
Texture::loadCubeMap("media/texture/cube/space/space");
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);
```

```
    LSdiffuseTexture =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_BaseC
olor1.png");
    LSnormalMap =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_Norm
al.png");

    WorkbenchDiffuseMap =
Texture::loadTexture("media/texture/workbench.png");

    skyboxShader.use();
    skyboxShader.setUniform("FogColour", fogColour);

    bladeEmissive.use();
    bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
    bladeEmissive.setUniform("Intensity", 8.0f);
    bladeEmissive.setUniform("RimPower", 2.5f);
    bladeEmissive.setUniform("RimIntensity", 0.6f);

    fogColour = bladeColours[bladeColourIndex];
    prog.use();
    prog.setUniform("Fog.Colour", fogColour);

    updateFogColour();
}

void SceneBasic_Uniform::compile()
{
    try {
        prog.compileShader("shader/basic_uniform.vert");
        prog.compileShader("shader/basic_uniform.frag");
        prog.link();
        prog.use();

        bladeEmissive.compileShader("shader/blade.vert");
        bladeEmissive.compileShader("shader/blade.frag");
        bladeEmissive.link();
        bladeEmissive.use();

        skyboxShader.compileShader("shader/skybox.vert");
        skyboxShader.compileShader("shader/skybox.frag");
        skyboxShader.link();
        skyboxShader.use();
    }
    catch (GLSLProgramException& e) {
        cerr << e.what() << endl;
        exit(EXIT_FAILURE);
    }
}

void SceneBasic_Uniform::update(float t)
{
    float deltaT = t - tPrev;
    if (tPrev == 0.0f)
        deltaT = 0.0f;
    tPrev = t;
    angle += 0.25f * deltaT;
    if (angle > glm::two_pi<float>())
        angle -= glm::two_pi<float>();

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        //blade toggle
        if (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS &&
!Q_Pressed) {
            bladeOn = !bladeOn;
            updateFogColour();

        }
        Q_Pressed = (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS);
        // colour cycling
        if (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS && !E_Pressed)
        {
            bladeColourIndex = (bladeColourIndex + 1) %
(int)bladeColours.size();

            bladeEmissive.use();
            bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);

            updateFogColour();
        }
        E_Pressed = (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS);
```

```
        //fog stuff
        if (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS && !F_Pressed)
        {
            fogEnabled = !fogEnabled;
        }
        F_Pressed = (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS);
        //rotate hilt
        if (glfwGetKey(win, GLFW_KEY_W) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_S) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_UP) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_DOWN) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;

        if (hiltYawDeg > 360.0f) hiltYawDeg -= 360.0f;
        if (hiltYawDeg < 0.0f)   hiltYawDeg += 360.0f;
    }
    render();

}

void SceneBasic_Uniform::render()
{

    prog.use();

    //glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT |
GL_STENCIL_BUFFER_BIT);

    // BlinnPhong spotlight
    glm::vec4 lightPosWorld = glm::vec4(10.0f * cos(angle), 10.0f, 10.0f *
sin(angle), 1.0f);
    prog.setUniform("Spot.Position", glm::vec3(view * lightPosWorld));

    glm::vec3 targetWorld = glm::vec3(0.0f, 2.0f, 3.0f);
    glm::vec3 lightDirWorldSpot = glm::normalize(targetWorld -
glm::vec3(lightPosWorld));

    glm::vec3 lightDirView = glm::mat3(view) * lightDirWorldSpot;
    prog.setUniform("Spot.Direction", lightDirView);


    glm::vec3 lightDirWorld = glm::normalize(glm::vec3(-0.6f, -1.0f,
-0.4f));
    glm::vec3 lightDirViewSpace = glm::mat3(view) * -lightDirWorld;
    prog.setUniform("Light.Position", glm::vec4(lightDirViewSpace, 0.0f));


    prog.setUniform("FogEnabled", fogEnabled ? 1 : 0);
    prog.setUniform("FogScale", fogScale);

    // Skybox stuff
    skyboxShader.use();
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);
    skyboxShader.setUniform("SkyboxTex", 0);

    glm::mat4 skyView = glm::mat4(glm::mat3(view));
    glm::mat4 skyMVP = projection * skyView;
    skyboxShader.setUniform("MVP", skyMVP);

    skyboxShader.setUniform("FogEnabled", fogEnabled ? 1 : 0);
    skyboxShader.setUniform("FogColour", fogColour);
    skyboxShader.setUniform("SkyFogAmount", skyFogAmount);

    sky.render();




    prog.use();
    //// Workbench
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, WorkbenchDiffuseMap);

    prog.setUniform("Material.Ks", vec3(0.0f));
    prog.setUniform("Material.Ka", vec3(0.1f));
    prog.setUniform("Material.Shininess", 180.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(0.0f, -1.0f, 0.0f));
    model = glm::rotate(model, glm::radians(25.0f), vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, vec3(0.5f));
    setMatrices();
```

```
    plane.render();

    // Lightsaber hilt
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, LSdiffuseTexture);

    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, LSnormalMap);

    prog.setUniform("Material.Ks", vec3(0.25f));
    prog.setUniform("Material.Ka", vec3(0.15f));
    prog.setUniform("Material.Shininess", 80.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(-1.0f, 2.0f, 2.0f));
    model = glm::scale(model, vec3(1.5f));
    model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
    model = glm::rotate(model, glm::radians(300.0f), vec3(0.0f, 0.0f, 1.0f));
    model = glm::rotate(model, glm::radians(hiltYawDeg), vec3(0.0f, 1.0f,
0.0f));

    setMatrices();
    LightsaberMesh->render();


    // Blade
    if (bladeOn)
    {
        glm::mat4 bladeBase = mat4(1.0f);
        bladeBase = glm::translate(bladeBase, vec3(10.0f, 2.1f, -4.0f));
        bladeBase = glm::rotate(bladeBase, glm::radians(270.0f), vec3(1.0f,
0.0f, 0.0f));
        bladeBase = glm::rotate(bladeBase, glm::radians(300.0f), vec3(0.0f,
0.0f, 1.0f));

        // -------- Pass 1: White core (writes stencil = 1 where the blade is)
--------
        bladeEmissive.use();

        glDisable(GL_BLEND);
        glEnable(GL_DEPTH_TEST);
        glDepthMask(GL_TRUE);

        // Stencil: always write 1 where fragments pass depth test
        glStencilMask(0xFF);
        glStencilFunc(GL_ALWAYS, 1, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);

        model = bladeBase;
        model = glm::scale(model, vec3(0.4f));
        setMatricesEmiss();

        bladeEmissive.setUniform("BladeColour", glm::vec3(1.0f));
        bladeEmissive.setUniform("Intensity", 10.0f);
        bladeEmissive.setUniform("RimPower", 5.0f);
        bladeEmissive.setUniform("RimIntensity", 0.25f);

        bladeEmissive.setUniform("Alpha", 1.0f);
        bladeEmissive.setUniform("CoreAlpha", 1.0f);

        BladeMEsh->render();

        // -------- Pass 2: Coloured glow shell (only OUTSIDE the core
silhouette) --------
        glEnable(GL_BLEND);
        glBlendFunc(GL_ONE, GL_ONE);

        // Don't write depth for glow, but DO depth test so hilt blocks it
        glDepthMask(GL_FALSE);

        // Stencil: only draw where stencil != 1 (i.e., outside the core)
        glStencilMask(0x00);
        glStencilFunc(GL_NOTEQUAL, 1, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);

        // Optional but usually improves the "wrap" look:
        // render only backfaces of the enlarged shell so it appears as an
outline
        glEnable(GL_CULL_FACE);
        glCullFace(GL_FRONT);

        model = bladeBase;
        model = glm::scale(model, vec3(0.4f * 1.4f));
        setMatricesEmiss();

        bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
```

```
        bladeEmissive.setUniform("Intensity", 2.5f);
        bladeEmissive.setUniform("RimPower", 2.0f);
        bladeEmissive.setUniform("RimIntensity", 1.0f);

        bladeEmissive.setUniform("Alpha", 0.55f);
        bladeEmissive.setUniform("CoreAlpha", 0.0f);

        BladeMEsh->render();

        // -------- Restore state --------
        glCullFace(GL_BACK);
        glDisable(GL_CULL_FACE);

        glDepthMask(GL_TRUE);
        glDisable(GL_BLEND);

        glStencilMask(0xFF);
        glStencilFunc(GL_ALWAYS, 0, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
    }




}

void SceneBasic_Uniform::setMatrices() {
    mat4 mv = view * model;
    prog.setUniform("ModelViewMatrix", mv);
    prog.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]), vec3(mv[1]),
vec3(mv[2])));
    prog.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::setMatricesEmiss() {
    mat4 mv = view * model;
    bladeEmissive.setUniform("ModelViewMatrix", mv);
    bladeEmissive.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]),
vec3(mv[1]), vec3(mv[2])));
    bladeEmissive.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::updateFogColour()
{
    fogColour = bladeOn ? bladeColours[bladeColourIndex] : fogGrey;

    // Update main scene shader fog colour
    prog.use();
    prog.setUniform("Fog.Colour", fogColour);

    // Update skybox fog colour too (so it stays consistent)
    skyboxShader.use();
    skyboxShader.setUniform("FogColour", fogColour);
}

void SceneBasic_Uniform::resize(int w, int h) {
    glViewport(0, 0, w, h);
    width = w;
    height = h;
    projection = glm::perspective(glm::radians(70.0f), (float)w / h, 0.3f,
100.0f);
}
```

what vert and frag files would you like to see to assist with this? im perfectly happy to have the edge detection ONLY on the lightsaber hilt (which i have setup with prog.)

Thought for 21s ›

Edge detection is *doable* in a "contained multi-pass section" like your blade glow **as long as you're OK with it being a small post-process** (Sobel-style) done on an offscreen texture.

If you keep it **ONLY on the hilt**, the scope is pretty reasonable:

- **Pass A (hilt mask/normal pass)**: render *just the hilt* into a small FBO texture (colour can be black, what matters is you write something useful per-pixel like **view-space normals** or even just a flat colour mask).
- **Pass B (edge detect pass)**: full-screen quad over the *main framebuffer*, sampling that hilt texture and doing Sobel to output an outline, blended on top.

That keeps it self-contained like your blade section and doesn't require rewriting the whole renderer.

What shader files I'd want to see

To wire this in cleanly with your current setup, these are the most useful:

1. `shader/basic_uniform.vert`

2. `shader/basic_uniform.frag`

Reason: I want to see what varyings you already output (Normal, Position, TexCoord etc.) and *what space* they're in. That determines whether we can quickly make a "hilt normal output" variant with minimal changes.

I **don't** need `blade.vert/frag` for this.

The fastest approach (minimal disruption)

1) Add a "hilt edge FBO"

Create an FBO with:

- `GLuint hiltFbo;`
- `GLuint hiltTex;` (RGBA8 is fine)
- `GLuint hiltDepth;` (depth renderbuffer is fine)

You render the hilt into this FBO with depth test enabled so the silhouette is correct.

2) Add a tiny hilt "mask/normal output" shader

Two options (pick based on time):

Option 1 (fastest): mask-only

- Render hilt as solid white into `hiltTex`
- Edge detect on intensity changes (works well for silhouette edges, not internal creases)

Option 2 (better edges): normal output

- Output encoded **view-space normal** to `hiltTex`
- Edge detect on normal discontinuities (gives nicer "toon-ish" edges)

Given you're short on time, I'd start with **mask-only**, then upgrade to normals if you have time.

3) Full-screen quad edge shader

A simple quad pass that samples `hiltTex` and applies Sobel. Output an outline colour with alpha, then:

```
C++
```

How it fits your `render()` structure

You'll end up with something like:

1. Render skybox/workbench/hilt as you already do (normal scene)
2. **Hilt edge section**
   - Bind `hiltFbo`, clear it
   - Render hilt only (mask/normal)
   - Bind default framebuffer
   - Draw fullscreen quad (edge detect) blended over scene
3. Blade section (or blade before edges — either works; if you want the hilt outline on top of everything, do edges last)

The only "new" shaders you'll need

You'll add two new shader programs:

Fullscreen quad vertex (`edge_quad.vert`)

- outputs UVs from a screen quad

Edge detect fragment (`edge_detect.frag`)

- samples `hiltTex`
- Sobel on luminance (mask) or Sobel on normals (normal map style)

I can write both of these immediately once I see whether your helper framework already has a quad mesh / screen-aligned triangle utility (sometimes labs include one, sometimes not).

Quick heads-up: the "bad idea" version

Trying to do "true edge detection" *directly while rendering the hilt* (without an FBO) is usually a pain, because edge detection is inherently neighbourhood-based (needs adjacent pixels). Doing it per-fragment in the hilt shader without a screen texture doesn't really work.

The stencil-outline trick (render scaled backfaces in black) is the *absolute fastest outline*, but it's not edge detection—it's more like what you're already doing on the blade glow shell.

Send me these and I'll tailor the exact code + shaders

Paste:

- `shader/basic_uniform.vert`
- `shader/basic_uniform.frag`

And if you have it, the bit of your framework that draws meshes / sets VAOs is enough to decide whether we do:

- a simple screen quad VAO you build once, or
- a "fullscreen triangle" (even simpler, no UV seam issues)

Once I can see the basic shader's inputs/outputs, I'll give you:

- the **2 new shaders** (quad vert + edge frag),
- the **FBO setup code** (init + resize),
- and the **exact render() insertion** written to match your style (like your blade multi-pass block).

basic_uniform.frag:

```
#version 460

in vec3 SpotDir;
in vec3 LightDir;
in vec3 ViewDir;
in vec2 TexCoord;
in vec3 ViewPos;

layout (location = 0) out vec4 FragColor;
layout (binding = 0) uniform sampler2D DiffuseTex;
layout (binding = 1) uniform sampler2D NormalMapTex;
layout (binding = 2) uniform sampler2D DiffuseTex2;


uniform struct MaterialInfo{
    vec3 Kd;
    vec3 Ka;
    vec3 Ks;
    float Shininess;
}Material;

uniform struct SpotLightInfo{
    vec3 L;
    vec3 La;
    vec3 Position;
    vec3 Direction;
    float Exponent;
    float Cutoff;
}Spot;

uniform struct LightInfo{
    vec4 Position;
    vec3 La;
    vec3 L;
}Light;

uniform struct FogInfo{
    float MaxDistance;
    float MinDistance;
    vec3 Colour;
}Fog;

//const int levels = 5;
//const float scaleFactor = 1.0/levels;

// Fog controls (simple + robust)
uniform int   FogEnabled;   // 0 = off, 1 = on
uniform float FogScale;     // 1.0 = normal, >1 stronger fog, <1 weaker


vec3 blinnPhong(vec3 n) {
    vec3 diffuse = vec3(0.0f);
    vec3 spec = vec3(0.0f);

    vec3 texColour = texture(DiffuseTex, TexCoord).rgb;

    vec3 ambient = Light.La * texColour;
    vec3 s = normalize(LightDir);
    //vec3 s = normalize(Light.Position.xyz - position);

    float sDotN = max(dot(s, n), 0.0);

    diffuse = texColour * sDotN;

    if (sDotN > 0.0) {
        vec3 v = normalize(ViewDir);
        vec3 h = normalize(v + s);
        spec = Material.Ks * pow(max(dot(h, n), 0.0), Material.Shininess);
    }
```

```
    return ambient + (diffuse + spec) * Light.L;
}



vec3 blinnPhongSpot(vec3 n)
{
    vec3 texColour = texture(DiffuseTex, TexCoord).rgb;

    vec3 ambient = Spot.La * texColour;

    vec3 s = normalize(LightDir);  // tangent-space light dir
    vec3 v = normalize(ViewDir);

    float cosAng = dot(normalize(-s), normalize(SpotDir));

    float outer = cos(Spot.Cutoff);                // outer cone edge
    float inner = cos(Spot.Cutoff * 0.85);         // inner cone edge

    float spotEdge = smoothstep(outer, inner, cosAng);
    float spotCore = pow(max(cosAng, 0.0), Spot.Exponent);
    float spotScale = spotEdge * spotCore;

    if (spotScale <= 0.001)
        return ambient;

    float sDotN = max(dot(s, n), 0.0);
    vec3 diffuse = texColour * sDotN;

    vec3 spec = vec3(0.0);
    if (sDotN > 0.0) {
        vec3 h = normalize(v + s);
        spec = Material.Ks * pow(max(dot(h, n), 0.0), Material.Shininess);
    }

    return ambient + spotScale * Spot.L * (diffuse + spec);
}

float fogFactorLinear(float dist)
{
    // 1 = fully shaded, 0 = fully fog
    float f = (Fog.MaxDistance - dist) / (Fog.MaxDistance -
Fog.MinDistance);
    return clamp(f, 0.0, 1.0);
}

void main()
{
    vec3 normal = texture(NormalMapTex, TexCoord).xyz;
    normal = normalize(normal * 2.0 - 1.0);

    vec3 litColour = blinnPhongSpot(normal);

    if (FogEnabled != 0)
    {
        // Use view-space distance; scale lets you tune “strength” easily
        float dist = length(ViewPos) * max(FogScale, 0.0001);

        float f = fogFactorLinear(dist);
        vec3 finalColour = mix(Fog.Colour, litColour, f);

        FragColor = vec4(finalColour, 1.0);
    }
    else
    {
        FragColor = vec4(litColour, 1.0);
    }
}

basic_uniform.vert:
#version 460

layout (location = 0) in vec3 VertexPosition;
layout (location = 1) in vec3 VertexNormal;
layout (location = 2) in vec2 VertexTexCoord;
layout (location = 3) in vec4 VertexTangent;

out vec3 SpotDir;
out vec3 LightDir;
out vec3 ViewDir;
out vec2 TexCoord;
out vec3 ViewPos;

uniform struct LightInfo{
    vec4 Position;
    vec3 La;
    vec3 L;
```

```
}Light;

uniform struct SpotLightInfo{
    vec3 L;
    vec3 La;
    vec3 Position;
    vec3 Direction;
    float Exponent;
    float Cutoff;
}Spot;

uniform mat4 ModelViewMatrix;
uniform mat3 NormalMatrix;
uniform mat4 ProjectionMatrix;
uniform mat4 MVP;

out vec3 LightIntensity, n, pos;

void main()
{
    vec3 N = normalize(NormalMatrix * VertexNormal);
    vec3 T = normalize(NormalMatrix * vec3(VertexTangent.xyz));

    T = normalize(T - N * dot(N, T));

    vec3 B = cross(N, T) * VertexTangent.w;

    // Columns are T, B, N
    mat3 TBN = mat3(T, B, N);
    mat3 invTBN = transpose(TBN);

    vec3 Position = (ModelViewMatrix * vec4(VertexPosition, 1.0)).xyz;
    ViewPos = Position;

    vec3 lightDir;
    if (Light.Position.w == 0.0)
        lightDir = normalize(Light.Position.xyz);      // directional
    else
        lightDir = Light.Position.xyz - Position;       // point light
    LightDir = invTBN * lightDir;

    ViewDir = invTBN * normalize(-Position);
    SpotDir = invTBN * normalize(Spot.Direction);

    TexCoord = VertexTexCoord;

    gl_Position = MVP * vec4(VertexPosition, 1.0);
}
```

Thought for 19s ›

Nice — your `basic_uniform` shaders are *perfectly fine* to leave alone. For a "quick edge detection only on the hilt" implementation, the fastest route is:

1. **Render the hilt into a tiny offscreen texture as a white mask** (black background)
2. **Run a Sobel edge detect full-screen pass** sampling that mask, and blend the outline over your main scene

This keeps it contained like your blade multi-pass block and doesn't mess with your existing lighting/fog/normal maps.

Below is a complete, drop-in style setup (C++ + 2 small shaders).

---

1) Add new members to `SceneBasic_Uniform` (header)

Add:

```
// --- Edge detect (hilt only) ---
GLSLProgram hiltMaskProg;
GLSLProgram edgeProg;

GLuint hiltFbo = 0;
GLuint hiltMaskTex = 0;
GLuint hiltDepthRb = 0;

GLuint fsTriVao = 0;   // fullscreen triangle VAO
bool edgeEnabled = true; // optional toggle if you want

// tweakables
float edgeThreshold = 0.25f;   // higher = fewer edges
```

C++

```
glm::vec3 edgeColour = glm::vec3(0.0f); // black outline
```

---

2) Compile the new shaders (your `compile()`)

Add two extra programs:

```cpp
hiltMaskProg.compileShader("shader/hilt_mask.vert");
hiltMaskProg.compileShader("shader/hilt_mask.frag");
hiltMaskProg.link();

edgeProg.compileShader("shader/edge_tri.vert");
```

(Keep your existing ones as-is.)

---

3) Create the FBO + fullscreen triangle (init + resize)

A) Fullscreen triangle VAO (init once)

In `initScene()` after `compile()`:

```cpp
```

B) FBO creation helper

Add this function:

```cpp
void SceneBasic_Uniform::initHiltEdgeFbo(int w, int h)
{
    // Clean up old
    if (hiltMaskTex) glDeleteTextures(1, &hiltMaskTex);
    if (hiltDepthRb) glDeleteRenderbuffers(1, &hiltDepthRb);
    if (hiltFbo) glDeleteFramebuffers(1, &hiltFbo);

    // Colour mask texture
    glGenTextures(1, &hiltMaskTex);
    glBindTexture(GL_TEXTURE_2D, hiltMaskTex);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA8, w, h, 0, GL_RGBA, GL_UNSIGNED_BYTE, nullptr);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);

    // Depth buffer
    glGenRenderbuffers(1, &hiltDepthRb);
    glBindRenderbuffer(GL_RENDERBUFFER, hiltDepthRb);
    glRenderbufferStorage(GL_RENDERBUFFER, GL_DEPTH_COMPONENT24, w, h);

    // FBO
    glGenFramebuffers(1, &hiltFbo);
    glBindFramebuffer(GL_FRAMEBUFFER, hiltFbo);
    glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, hiltMaskTex, 0);
    glFramebufferRenderbuffer(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_RENDERBUFFER, hiltDepthRb);

    GLenum drawBufs[] = { GL_COLOR_ATTACHMENT0 };
    glDrawBuffers(1, drawBufs);

    GLenum status = glCheckFramebufferStatus(GL_FRAMEBUFFER);
    if (status != GL_FRAMEBUFFER_COMPLETE) {

    glBindFramebuffer(GL_FRAMEBUFFER, 0);
}
```

C) Call it from `resize()`

At the end of `resize(int w, int h)`:

```cpp
```

(That way it always matches your window.)

---

4) Add the edge passes inside `render()`

You already render the main scene + hilt + blade. We'll insert this **after you render the hilt normally** (so the outline overlays it).

Right after:

```cpp
```

Add:

```
if (edgeEnabled)
{
    // ---------- HILT EDGE PASS A: render hilt mask into FBO ----------
    glBindFramebuffer(GL_FRAMEBUFFER, hiltFbo);
    glViewport(0, 0, width, height);

    glDisable(GL_BLEND);
    glEnable(GL_DEPTH_TEST);
    glDepthMask(GL_TRUE);

    glClearColor(0, 0, 0, 0);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    hiltMaskProg.use();

    // Reuse the same model transform you used for the hilt
    // (IMPORTANT: copy/paste the same model matrix setup)
    model = mat4(1.0f);
    model = glm::translate(model, vec3(-1.0f, 2.0f, 2.0f));
    model = glm::scale(model, vec3(1.5f));
    model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
    model = glm::rotate(model, glm::radians(300.0f), vec3(0.0f, 0.0f, 1.0f));
    model = glm::rotate(model, glm::radians(hiltYawDeg), vec3(0.0f, 1.0f, 0.0f));

    // For the mask shader we only need MVP
    mat4 mv = view * model;
    hiltMaskProg.setUniform("MVP", projection * mv);
```

ChatGPT 5.2 Thinking

Share

```
    // ---------- HILT EDGE PASS B: Sobel detect over main framebuffer ----------
    glBindFramebuffer(GL_FRAMEBUFFER, 0);
    glViewport(0, 0, width, height);

    glDisable(GL_DEPTH_TEST);
    glDepthMask(GL_FALSE);

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

    edgeProg.use();
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, hiltMaskTex);
    edgeProg.setUniform("MaskTex", 0);

    edgeProg.setUniform("TexelSize", glm::vec2(1.0f / width, 1.0f / height));
    edgeProg.setUniform("Threshold", edgeThreshold);
    edgeProg.setUniform("Opacity", edgeOpacity);
    edgeProg.setUniform("Thickness", edgeThickness);
    edgeProg.setUniform("EdgeColour", edgeColour);

    glBindVertexArray(fsTriVao);
```

C++

```
    // Restore defaults
    glDisable(GL_BLEND);
    glDepthMask(GL_TRUE);
    glEnable(GL_DEPTH_TEST);
}
```

That's the "self-contained section" you wanted.

---

5) Add these 2 new shader files

`shader/hilt_mask.vert`

```
#version 460
layout (location = 0) in vec3 VertexPosition;

uniform mat4 MVP;

void main()
```

glsl

`shader/hilt_mask.frag`

```
#version 460
layout (location = 0) out vec4 FragColor;

void main()
{
```

glsl

`shader/edge_tri.vert` (fullscreen triangle)

```glsl
#version 460
out vec2 vUV;

void main()
{
    // Fullscreen triangle (no VBO)
    vec2 pos;
    if (gl_VertexID == 0) pos = vec2(-1.0, -1.0);
    if (gl_VertexID == 1) pos = vec2( 3.0, -1.0);
    if (gl_VertexID == 2) pos = vec2(-1.0,  3.0);

    gl_Position = vec4(pos, 0.0, 1.0);

}
```

`shader/edge_detect.frag` (Sobel on the mask)

```glsl
#version 460
in vec2 vUV;
layout (location = 0) out vec4 FragColor;

layout (binding = 0) uniform sampler2D MaskTex;

uniform vec2  TexelSize;   // (1/width, 1/height)
uniform float Threshold;   // edge cutoff
uniform float Opacity;     // 0..1
uniform float Thickness;   // 1..N
uniform vec3  EdgeColour;  // outline colour

float sampleMask(vec2 uv)
{
    return texture(MaskTex, uv).r; // 0..1 mask
}

void main()
{
    vec2 t = TexelSize * max(Thickness, 0.0001);

    // Sobel kernel on mask intensity
    float tl = sampleMask(vUV + vec2(-t.x,  t.y));
```

```glsl
    float  t0 = sampleMask(vUV + vec2( 0.0,  t.y));
    float  b0 = sampleMask(vUV + vec2( 0.0, -t.y));

    float tr = sampleMask(vUV + vec2( t.x,  t.y));
    float  r = sampleMask(vUV + vec2( t.x,  0.0));
    float br = sampleMask(vUV + vec2( t.x, -t.y));

    float gx = (-1.0*tl) + (1.0*tr)
             + (-2.0*l ) + (2.0*r )
             + (-1.0*bl) + (1.0*br);

    float gy = ( 1.0*tl) + (2.0*t0) + (1.0*tr)
             + (-1.0*bl) + (-2.0*b0) + (-1.0*br);

    float g = sqrt(gx*gx + gy*gy);

    // threshold -> alpha
    float a = smoothstep(Threshold, Threshold * 1.5, g) * Opacity;

    if (a <= 0.001)
        discard;

    FragColor = vec4(EdgeColour, a);
}
```

---

6) How to tweak the look quickly

These are the main knobs:

- `edgeThreshold`: **0.10–0.35** (lower = more edges)
- `edgeThickness`: **1.0–2.5** (thicker outline)
- `edgeOpacity`: **0.5–1.0**

If you want a "cartoon black outline", keep `edgeColour = vec3(0.0)`.

One important gotcha in your project

You have **stencil test enabled globally** (`glEnable(GL_STENCIL_TEST);` in `initScene()`), and your blade uses it.

For the edge passes, stencil usually won't matter, but if you ever see the outline mysteriously missing, add this inside the edge block:

```C++
glDisable(GL_STENCIL_TEST);
// ... edge passes ...
glEnable(GL_STENCIL_TEST);
```

Only do that if it actually misbehaves — I'm flagging it because stencil state bugs are common.

If you want the outline to include **internal creases** (not just silhouette), tell me and I'll switch the hilt FBO pass to output **view-space normals** instead of a white mask (still only 1 extra pass; the edge shader changes slightly).